

TollRoad — Unit Economics (All-In AWS Cost to Stream)

What does it actually cost to stream one minute of music on TollRoad? This is a bottom-up marginal cost across **every AWS service in the path**, expressed per **minute**, per **song**, and per **user**. All rates are us-east-1 on-demand, **verified 2026-06-12 against the live AWS Pricing API** (`aws pricing get-products` — exact queries in the appendix). Marginal = the incremental cost of one more minute (free tiers are a fixed subsidy, excluded from the per-unit rate; treated separately at the end).

Re-modeled 2026-06-18 for the API re-platform. The serving path is now **Vercel client → API Gateway (REST) → one tollroad-api Lambda → Aurora DSQL**, with audio still on **S3 → signed CloudFront** (one mp3 per track, not HLS). The bottom line barely moved — CloudFront egress still dominates — but the middle tier is re-composed: a new API Gateway + Lambda line replaces what used to be Vercel-side metering, the DynamoDB hot path shrank to a best-effort mirror, and Aurora DSQL is now the system of record. The per-minute total went from \$0.0001155 to **\$0.0001144**; the 10K-MAU bill from \$957 to **\$948 gross / ~\$846 net**.

The canonical streamed minute

One minute of playback consumes a fixed, known basket of AWS resources:

Dimension	Value	Why
Audio bitrate	160 kbps → 1.2 MB/min	Music-quality stream; bitrate is the dominant lever (see below)
CloudFront requests	~2 / min	one mp3 per track via signed URL; the browser pulls it in a handful of range GETs (was ~10/min under HLS)
<code>/v1/charge</code> calls	~1 / min	the metering hot path through API Gateway → Lambda; idempotent per <code>user#track#minute</code>
API Gateway requests	~1.7 / min	charge + signed-grant refresh + browse/library navigation, averaged over a streamed minute
Lambda invocations	~1.8 / min	one <code>tollroad-api</code> invoke per API request (256 MB, ~100 ms) + the rare rollup batch
Aurora DSQL	1 charge txn/min	<code>BEGIN → dup-check → debit balance → insert ledger → COMMIT</code> , the system of record
DynamoDB write units	~2 / min	a best-effort METER mirror: 1 conditional <code>PutItem</code> + 1 GSI1 (<code>ALL</code>) replica, 1/ min
DynamoDB stream reads	\$0	<code>GetRecords</code> by the rollup's Lambda event-source mapping are not billed
KMS	~0 calls/min	SSE-KMS + CloudFront cache → no per-play decrypt (<i>requires S3 Bucket Keys — see caveats</i>)

Metering is a dual-write. `domain/billing.ts` debits the wallet and appends the royalty ledger transactionally in DSQL (durable, authoritative); `domain/meter.ts` then mirrors a single METER event into DynamoDB, whose stream drives the rollup Lambda. Both key on the same `user#track#minute` idempotency key, so the mirror is at-

least-once-safe and reconciles to a no-op. DSQL is the record of truth; the DynamoDB half is best-effort and skipped entirely in local dev.

Cost per minute streamed — every AWS service

Shown as **\$ per 1,000 minutes** (the per-minute figures are fractions of a cent, so 1,000 min is the readable anchor).

AWS service	Unit math	\$ / 1,000 min	% of total
CloudFront — egress	1.2 GB × \$0.085/GB	0.10200	89.1%
API Gateway — REST requests	1,714 req × \$3.50/M	0.00600	5.24%
CloudFront — requests	2,000 req × \$0.01/10k	0.00200	1.75%
Aurora DSQL — compute (DPU)	~0.220 DPU/min × \$8/M (<i>modeled</i>)	0.00176	1.54%
DynamoDB — writes (METER mirror)	2,000 WRU × \$0.625/M	0.00125	1.09%
Lambda — API + rollup	~1,811 req + ~45 GB-s	0.00112	0.98%
KMS — key (amortized)	\$1/mo ÷ 8.17M min	0.00012	0.10%
S3 — origin GET + storage (amortized)	cache-miss reads only	0.00010	0.09%
DynamoDB — storage (event TTL)	0.3 KB/event, ~30-day resident	0.00006	0.05%
Aurora DSQL — storage (ledger)	100 B/min × \$0.33/GB-mo	0.00003	0.03%
DynamoDB — streams	Lambda consumer → GetRecords free	0.00000	0%
TOTAL		0.11444	100%

All-in marginal cost ≈ \$0.0001144 per minute ≈ 0.01144¢ / minute.

Where the money actually goes

- **CloudFront (egress + requests) = ~91%** of the marginal cost. TollRoad is, economically, a **bandwidth business** wearing a database's clothes.
- **The new serving tier — API Gateway + Lambda — is ~6%**, almost all of it API Gateway's per-request fee. It's the largest non-CloudFront line and it's new: under the old architecture this metering ran as Vercel functions, off the AWS bill.
- **The entire data plane — DynamoDB, Aurora DSQL, S3, KMS — is ~3%** combined. Per stream, **the databases are effectively free**; serverless + scale-to-zero means you pay for work done, and the work per minute is tiny.

Rolled up: per song, per user

Unit	Minutes	All-in AWS cost
Per minute	1	\$0.0001144 (0.0114¢)
Per song	3.0	\$0.000343 (0.034¢)
Per user / month	817	\$0.0935 (9.3¢)
Per user / year	9,800	\$1.121

So a fully-active listener — someone who streams as much as the average Spotify user — costs TollRoad about **\$1.12/year, all-in, across the entire AWS stack** — essentially unchanged by the re-platform.

Monthly AWS bill — the stack as deployed

The marginal numbers above, re-expressed as the actual monthly bill the deployed stack (`infra/lib/tollroad-stack.ts`) generates at three scales. Assumes a 10,000-track catalog (~36 GB at 160 kbps) and the always-free tiers (1 TB + 10M req CloudFront, 25 GB DynamoDB storage, 100K DPU + 1 GB-mo DSQL, 1M req + 400K GB-s Lambda). **API Gateway REST has no perpetual free tier** — its 1M-req credit expires after 12 months, so it's a real line from steady state.

Line	Demo (~0 traffic)	1,000 MAU (0.82M min/mo)	10,000 MAU (8.17M min/mo)
CloudFront egress	\$0	\$0 (0.98 TB ≤ 1 TB free)	\$748 (9.8 TB – 1 free, all in \$0.085 tier)
CloudFront requests	\$0	\$0 (1.6M ≤ 10M free)	\$6.34 (16.3M – 10M free)
API Gateway (REST)	~\$0 (12-mo free)	\$4.90 (1.4M req)	\$49.00 (14M req, no free tier)
DynamoDB writes	~\$0	\$1.02 (1.6M WRU)	\$10.21 (16.3M WRU)
DynamoDB reads + storage	~\$0	~\$0	~\$0 (reads removed; storage ≤ 25 GB free)
Aurora DSQL	\$0 (scale-to-zero)	\$0.64 (180K – 100K free)	\$13.60 (1.8M DPU – 100K free)
Lambda	\$0	\$0 (free tier)	\$2.76 (13.8M req; 370K GB-s ≤ 400K free)
KMS key	\$1	\$1	\$1
KMS decrypt	\$0	~\$0	\$12.26 (Bucket Keys OFF — see levers; →\$0 if ON)
S3 catalog storage + origin	~\$0.85	~\$0.90	~\$2.50
Total / month	≈ \$2	≈ \$9	≈ \$846 (≈ \$0.085/user; ~\$834 with Bucket Keys ON)

The 10K-MAU per-user figure (\$0.085) lands just under the marginal 9.3¢ — the gap is exactly the free-tier subsidy. **Until ~1,000 active users the entire AWS bill rounds to the \$1/mo KMS key, catalog storage, and a few dollars of API Gateway** (~\$9/mo at 1K MAU, vs. ~\$4 under the old Vercel-metered topology).

Fixed / amortized AWS lines (not per-minute)

These don't scale with minutes; they amortize to ~\$0 per stream but are listed for completeness:

Item	Cost	Per-stream impact
S3 audio storage	a 3-min song @ 160 kbps ≈ 3.6 MB × \$0.023/GB-mo = \$0.00008 / song / mo	divided across every stream of that song → negligible
KMS CMK	\$1 / mo per key <i>version</i> ; the stack enables annual rotation, so \$2 / mo from year 2 (AWS caps rotated keys at \$2)	\$0.00000012–24/min amortized (already in the table)
Aurora DSQL / Lambda idle	\$0 — both scale to zero	no standing charge
API Gateway idle	\$0 — pay-per-request, no hourly charge	only the per-request fee above
Bedrock "explain my statement"	~1.5k in × \$1.10/M + 0.4k out × \$5.50/M = \$0.00385 / call (Bedrock on-demand runs ~10% above the direct Anthropic API's \$1/\$5)	only when a user asks; ~+4% to a user's monthly cost if used once

Margin

At an illustrative listener price of **\$0.01/min**:

	Per minute	Per user / mo (817 min)
Listener pays	\$0.01000	\$8.170
All-in AWS cost	\$0.0001144	\$0.0935
AWS as % of price	1.14%	1.14%

AWS consumes ~1% of gross billed revenue. But **AWS is not the dominant infrastructure cost — payment processing is**. Both come out of the same platform take, and Stripe-style card fees run **3–7× the AWS bill**.

What the platform actually bears: AWS *and* inbound payments

The platform's cost is **AWS + the inbound processing fee on wallet top-ups**. Standard card processing is **2.9% + \$0.30** per charge; the fixed \$0.30 makes **top-up frequency** the biggest single lever. Per active listener (gross \$98/yr):

Platform cost line	\$ / user / yr	% of gross
AWS (all-in, marginal)	~\$1.12	~1.1%
Payments — card, monthly top-up	\$6.44	6.6%
Payments — card, annual top-up	\$3.14	3.2%
Payments — ACH-first wallet (0.8%, \$5 cap)	\$0.78	0.8%

Artist payout is *not* a platform cost. The artist earns the royalty (90% of gross); the fee to *disburse* it — Stripe Connect / ACH transfer, ~0.25% of payout volume plus any per-account or instant-payout fees — is deducted from the **artist's** share, not the platform rake. So the platform bears AWS + inbound payments only.

At a 10% platform take (\$9.80/user/yr of rake), the **kept margin after AWS + inbound payments** swings entirely on the rail and top-up size:

Wallet config	Rake kept
Card, monthly top-up	23%
Card, annual top-up	56%
ACH-first, \$10 min / auto-reload	80%
Pass-through (listener pays the fee)	88%

So the earlier "~88% platform margin" was **AWS-only and optimistic**. All-in, the platform keeps **23–88% of the rake** — and at a 6% rake with monthly card top-ups it goes **negative**. The rake floor is set by **payments, not AWS**. The fix is a **prepaid wallet with a \$10 minimum on ACH rails** (bank debit caps the fee), with cards as the instant fallback — see [COST VISUAL.pdf](#) pp. 1, 6–7.

Cost levers (in order of impact)

1. **Bitrate** — egress is 89% of cost and scales linearly. 160 → 128 kbps ≈ –18% all-in; 96 kbps ≈ –35%. Quality tiers; default mobile lower.
2. **API Gateway: REST → HTTP API** — the gateway is `apigw.RestApi` at \$3.50/M; an HTTP API is **\$1.00/M** (`USE1-ApiGatewayHttpRequest`), cutting the new ~\$49/mo line to ~\$14/mo (–\$35/mo at 10K MAU). Highest-ROI change after bitrate, now that API Gateway is the top non-CloudFront line.

3. **S3 Bucket Keys** — collapse per-object KMS requests ~99%, pinning KMS at the flat key fee. **Not yet enabled in the stack** (`bucketKeyEnabled` is unset on the audio bucket) — without it, every cache-miss GET is a billed `kms:Decrypt` (at 10K MAU and a 5% miss rate, ~4M calls ≈ **+\$12/mo**). One-line CDK fix.
4. **Edge cache-hit ratio** — long TTLs on immutable audio keep S3 origin GETs and KMS calls near zero; popular catalog is served entirely from the edge.
5. **Charge cadence** — `/v1/charge` runs ~once per streamed minute. Fewer, coarser charges (e.g. settle every 2–3 min) would shave the API Gateway, Lambda, DSQL-txn, and DynamoDB-mirror lines proportionally — at the cost of meter granularity.
6. **Egress volume tiers** — CloudFront drops to \$0.08/GB past 10 TB/mo and as low as \$0.02/GB at PB scale, so the dominant line *deflates* as the business grows.
7. **Drop the DynamoDB mirror** — DSQL is already the system of record; the METER mirror + Streams + rollup Lambda exist only to feed per-artist/day summaries. Computing those directly in DSQL would retire the ~\$10/mo DynamoDB-write line and the whole stream/rollup path. The GSI1 (`ALL`) projection is the bulk of the mirror's WRU; `KEYS_ONLY` would halve it if the mirror stays.

Margin levers (bigger than all of the above):

1. **Payment rail** — ACH/bank debit caps the fee (Stripe 0.8% → \$5 ceiling; Adyen \$0.26 flat) vs. cards' uncapped 2.9% + \$0.30. For a wallet, ACH-first ≈ 4× cheaper than cards → +58 pts of kept rake.
2. **Top-up size** — the fixed \$0.30/charge means fewer, larger top-ups win. Monthly → annual card top-ups: \$6.44 → \$3.14/user/yr. Enforce a **\$10 minimum** (keeps even the card fallback under ~6%).
3. **Avoid Stripe metered (per-user monthly invoicing)** — it maxes the fixed-fee count *and* adds the 0.5% Billing fee → 7.1% of gross, the worst option. Keep metering internal (DSQL/DynamoDB); settle aggregates only.

Assumptions & caveats

- **Bitrate 160 kbps / 1.2 MB per minute and 3.0 min average song** (consistent with 9,800 min/yr ÷ 3,278 songs ≈ 3 min). Both are direct linear scalars — adjust and every figure moves proportionally.
- **All unit rates verified 2026-06-12 via the AWS Pricing API** (us-east-1 on-demand): CloudFront \$0.085/GB + \$0.01/10k HTTPS; API Gateway REST \$3.50/M (HTTP \$1.00/M); DynamoDB \$0.625/M WRU, \$0.25/GB-mo; DSQL \$8/M DPU, \$0.33/GB-mo; Lambda \$0.20/M + \$16.67/M GB-s; KMS \$1/key-version-mo + \$0.03/10k; S3 \$0.023/GB-mo + \$0.40/M GET; Bedrock Haiku 4.5 \$1.10/M in, \$5.50/M out.
- **Serving path**: Vercel is a pure client; every API call proxies to **API Gateway (REST) → one tollroad-api Lambda (256 MB) → Aurora DSQL**. Modeled at ~14M API requests/mo at 10K MAU — ~8.2M `/v1/charge` (1/min) + grant refreshes + browse/library navigation. This load was **off the AWS bill before the re-platform** (it ran as Vercel functions).
- **mp3, not HLS**: audio is one mp3 per track (`audio/<id>.mp3`) behind a signed CloudFront URL. Egress is identical to HLS (1.2 MB/min), but CloudFront *requests* drop from ~10/min (segments) to ~2/min (range GETs). This is the **softest assumption** — a chattier player (more range GETs/seek) pushes the request line back up; at ~10/min it returns to ~\$80/mo and the total lands near \$1,013. Egress dominates either way.
- **Metering write path**: `/v1/charge` writes the authoritative balance debit + ledger insert in one DSQL transaction (`domain/billing.ts`), then mirrors one METER event to DynamoDB (`domain/meter.ts`). Hence ~2 WRU per streamed minute in DynamoDB (1 item + 1 GSI1 `ALL` replica), at 1/min cadence — down from ~4 WRU/min under the old DynamoDB-balance model. **No DynamoDB reads remain** in the backend (the dup-check is a DSQL query), so the old live-meter RRU line is gone.
- **DynamoDB Streams cost \$0 here**: the only consumer is the rollup Lambda event-source mapping, whose GetRecords calls AWS does not bill. The \$0.02/100k rate applies only to non-Lambda consumers.
- **Lambda is now the API tier, not just the rollup**: ~14.8M invocations/mo (one per API request + rare rollup batches), 256 MB, ~100 ms avg → ~370K GB-s, which sits *under* the 400K GB-s perpetual free tier, so net duration is \$0 (gross \$6.17).
- **Aurora DSQL DPU-per-minute is modeled, not measured** (~0.220 DPU/min, ≈3× the old rollup-only figure to cover the synchronous charge transaction + catalog/library reads). Validate against `AWS/AuroraDSQL TotalDPU` CloudWatch metrics under real load. Even at 5× the assumed DPU, DSQL stays under 3% of all-in cost — the conclusion (databases ≈ free per stream) is robust.

- Figures are **marginal** (cost of one more minute). Perpetual free tiers (1 TB CloudFront egress + 10M requests, 25 GB DynamoDB, 100K DSQL DPU + 1 GB-mo, 1M Lambda req + 400K GB-s, 20K KMS requests) make the **first slice of traffic effectively free** — at low volume the real bill rounds to the **\$1/mo KMS key** plus a few dollars of un-subsidized API Gateway (see the monthly-bill table).

Appendix — how the rates were verified

All prices pulled live from the AWS Pricing API (us-east-1 endpoint), e.g.:

```
# DynamoDB on-demand / storage / streams
aws pricing get-products --region us-east-1 --service-code AmazonDynamoDB \
  --filters "Type=TERM_MATCH,Field=location,Value=US East (N. Virginia)"

# API Gateway — REST ($3.50/M) vs HTTP ($1.00/M) request pricing
aws pricing get-products --region us-east-1 --service-code AmazonApiGateway \
  --filters "Type=TERM_MATCH,Field=location,Value=US East (N. Virginia)"
# REST → usagetype USE1-ApiGatewayRequest, operation ApiGatewayRequest
# HTTP → usagetype USE1-ApiGatewayHttpRequest

# Aurora DSQL (DPU + storage)
aws pricing get-products --region us-east-1 --service-code AuroraDSQL \
  --filters "Type=TERM_MATCH,Field=location,Value=US East (N. Virginia)"

# CloudFront egress tiers + request pricing
aws pricing get-products --region us-east-1 --service-code AmazonCloudFront \
  --filters "Type=TERM_MATCH,Field=productFamily,Value=Data Transfer" \
    "Type=TERM_MATCH,Field=fromLocation,Value=United States"

# Lambda, S3, KMS — service codes AWSLambda, AmazonS3, awskms (same pattern)

# Bedrock Claude Haiku 4.5 (newer models live under FoundationModels)
aws pricing get-products --region us-east-1 \
  --service-code AmazonBedrockFoundationModels \
  --filters "Type=TERM_MATCH,Field=regionCode,Value=us-east-1"
```

Key dimensions returned (2026-06-12): DDB WriteRequestUnits \$0.625/M, Streams \$0.0000002/read-request-unit (free for Lambda pollers); API Gateway USE1-ApiGatewayRequest \$0.0000035/req (REST), USE1-ApiGatewayHttpRequest \$0.000001/req (HTTP); DSQL USE1-DSQL-DistributedProcessingUnits \$0.000008/DPU, Storage \$0.33/GB-Mo; CloudFront 0-10 TB \$0.085/GB, HTTPS \$0.000001/req; KMS \$1/key-version-mo; Bedrock Claude Haiku 4.5: \$1.10/M input, \$5.50/M output.